



GUJARAT TECHNOLOGICAL UNIVERSITY

Shree Swami Atmanand Saraswati Institute of Technology



A

Micro Project Report on

“WEATHER DATA TRACKER”

Under subject of

Python For Data Science(BE05000231)

B.E 3rd year, semester – V

COMPUTER ENGINEERING

Submitted by group:

Sr. No.	Name	Enrollment No.
1	Kukadiya Zisha Pankajbhai	240760107099
2	Lakkad Jeeya Devang	240760107102
3	Lathiya Era Sanket	240760107103
4	Maniya Yashasvi Ishwarbhai	240760107116
5	Meruliya Ami Mansukhbhai	240760107123

CERTIFICATE

This is to certify that the Project entitled "Weather Data Tracker" has been prepared by **Kukadiya Zisha Pankajbha (240760107099)**, **Lakkad Jeeya Devang (240760107102)**, **Lathiya Era Sanket (240760107103)**, **Maniya Yashasvi Ishwarbhai (240760107116)**, **Meruliya Ami Mansukhbhai (240760107123)** in the subject of **Web Application Development (BE05000281)** under my guidance in partial fulfilment of the degree of **Bachelor of Engineering in Computer Engineering Department (5th Semester)** of Gujarat Technological University, Ahmedabad during the academic year 2026-2027.

Date: / /

Place: SSASIT



Project Guide
(Prof. Ami D. Patel)



Head of the Department
(Prof. Chirag R. Patel)

ABSTRACT

The **Weather Data Tracker** is a web-based mini project that allows users to search, view, store, and analyse weather data for different cities. It displays parameters such as temperature, humidity, pressure, wind speed, visibility, and forecasts, along with weather history, favourite cities, unit conversion, and analytics. The project uses web technologies such as **HTML, CSS, JavaScript, Bootstrap, Python, Flask, SQLite, Pandas, NumPy, and Plotly** to provide an interactive and user-friendly weather tracking system.

Table of content

Sr. No.	Topic	Page No.
	Certificate	II
	Abstract	III
1	Introduction	1
2	Problem Statement	2
3	Objectives	3
4	Literature Survey	4
5	Existing System	5
6	Proposed System	6
7	Dataset Description	8
8	Dataset Attributes	9
9	Data Collection Process	10
10	Data Preprocessing	12
11	Methodology	14
12	System Design	16
13	System Architecture	18
14	Exploratory Data Analysis	20
15	Results and Discussion	21
16	Future Scope	22
	Conclusion	23

1. INTRODUCTION

Weather information is used in daily planning, transportation, agriculture, tourism and many other activities. Modern web applications can make weather information easier to access by combining an external weather service, a responsive user interface and data-analysis features.

The Weather Data Tracker is developed as a Computer Engineering mini project to demonstrate the integration of web development and data processing. The uploaded README describes a feature-rich application developed using Python 3, Flask, SQLite, SQLAlchemy, Pandas, NumPy, Plotly, Matplotlib, HTML5, CSS3, JavaScript and Bootstrap 5. The application is intended to allow users to search weather information for cities, view forecasts, maintain history and favourites, and analyse stored records.

The uploaded standalone HTML implementation provides the visible user interface. It contains a sidebar with Dashboard, 5-Day Forecast, Search History, Favourites, Analytics & Charts, and About & Viva Guide sections. The interface is responsive and uses a soft pastel weather theme.

The major functional areas of the application are:

- City search and current weather dashboard
- 5-day forecast
- Weather search history
- Favourite cities
- Analytics and interactive charts
- Celsius/Fahrenheit unit switching
- Offline demo fallback for presentations

2. PROBLEM STATEMENT

Weather information is often available from different websites and applications, but users may need to repeatedly search for cities, compare values, remember previous searches and interpret trends manually.

A useful weather tracker should therefore provide a single interface for collecting weather observations and presenting them in a structured way. It should also handle cases where an API key is unavailable, or the network service cannot be used during a demonstration.

The problem addressed by the project can be stated as follows:

- Provide a simple interface for searching weather information by city.
- Display important current weather parameters in one dashboard.
- Provide a 5-day forecast view.
- Maintain a history of searched weather records.
- Allow users to save and remove favourite cities.
- Convert temperature values between Celsius and Fahrenheit.
- Calculate useful descriptive statistics from stored weather records.
- Present weather trends through charts and visual summaries.
- Provide an offline/demo mode when a live API key is not configured.

3. OBJECTIVES

Primary Objective:

To develop a responsive Weather Data Tracker web application that combines weather-data retrieval, storage, processing and visualisation in a single user-friendly system.

Specific Objectives:

- To search for weather information for different cities.
- To display temperature, humidity, pressure, wind and other weather parameters.
- To provide a 5-day forecast.
- To store weather-search history.
- To manage favourite cities.
- To support Celsius and Fahrenheit temperature units.
- To analyse historical weather records using Pandas and NumPy in the documented Python architecture.
- To generate interactive charts using Plotly.
- To provide an offline demo fallback for reliable presentation.
- To build a responsive interface for desktop, laptop, tablet and mobile browsers.
- To demonstrate practical integration of web development and Data Science concepts.

4. LITERATURE SURVEY

Weather applications commonly combine external weather services, web or mobile interfaces, historical storage and data visualisation. The literature-oriented design of this project focuses on four related areas: weather-data retrieval, historical data management, descriptive analytics and visualisation.

Study 1: Weather Data Retrieval

Modern weather applications commonly consume data from a weather API. A typical weather record contains a city name, temperature, humidity, atmospheric pressure, wind information, cloudiness and weather condition. The project README identifies OpenWeatherMap as the external API used by the documented full-stack architecture.

Study 2: Historical Weather Tracking

Historical storage makes it possible to review previous searches and compare weather observations. The README specifies SQLite as the database and identifies WeatherHistory and FavoriteCity as SQLAlchemy models. The standalone HTML implementation demonstrates the same user-facing concept with browser localStorage.

Study 3: Data Analysis

Pandas and NumPy are suitable for converting weather records into structured analytical data. Descriptive measures such as averages, minimums, maximums and counts can summarise stored observations and support trend analysis.

Study 4: Data Visualisation

Visualisation helps users understand changes in weather variables. The project uses Plotly for interactive charts in the documented architecture and includes temperature, humidity, wind and weather-condition visualisations in the standalone interface.

Table 4.1 - Literature Survey Summary

Area	Common Technique
Weather Retrieval	REST API / structured JSON
Historical Tracking	SQLite / browser storage
Data Processing	Pandas and NumPy
Trend Analysis	Descriptive statistics
Data Presentation	Interactive charts
Responsive Interface	HTML5, CSS3, Bootstrap, JavaScript

5. EXISTING SYSTEM

A basic weather-search system normally focuses on displaying current conditions after a user enters a city. Such systems may provide limited history, limited comparison facilities and little analytical support.

Typical limitations of a basic weather-search approach include:

- Repeated manual searches for the same cities
- No organised local history of previous searches
- Limited favourite-city management
- Limited statistical analysis
- Few interactive visualisations
- Dependence on continuous API availability during demonstrations

RESEARCH GAP

The project addresses the gap between a simple weather lookup interface and a more complete weather-data tracking and analysis application. The proposed system adds history, favourites, unit conversion, forecast presentation, descriptive analytics and interactive visualisation to the weather lookup process.

6. PROPOSED SYSTEM

The proposed Weather Data Tracker combines a responsive web interface with weather-data retrieval and an analytical layer. According to the README, the full-stack design uses Flask as the backend, SQLite/SQLAlchemy for storage, a WeatherService for OpenWeatherMap requests Pandas/NumPy/Plotly utilities for analysis and visualisation.

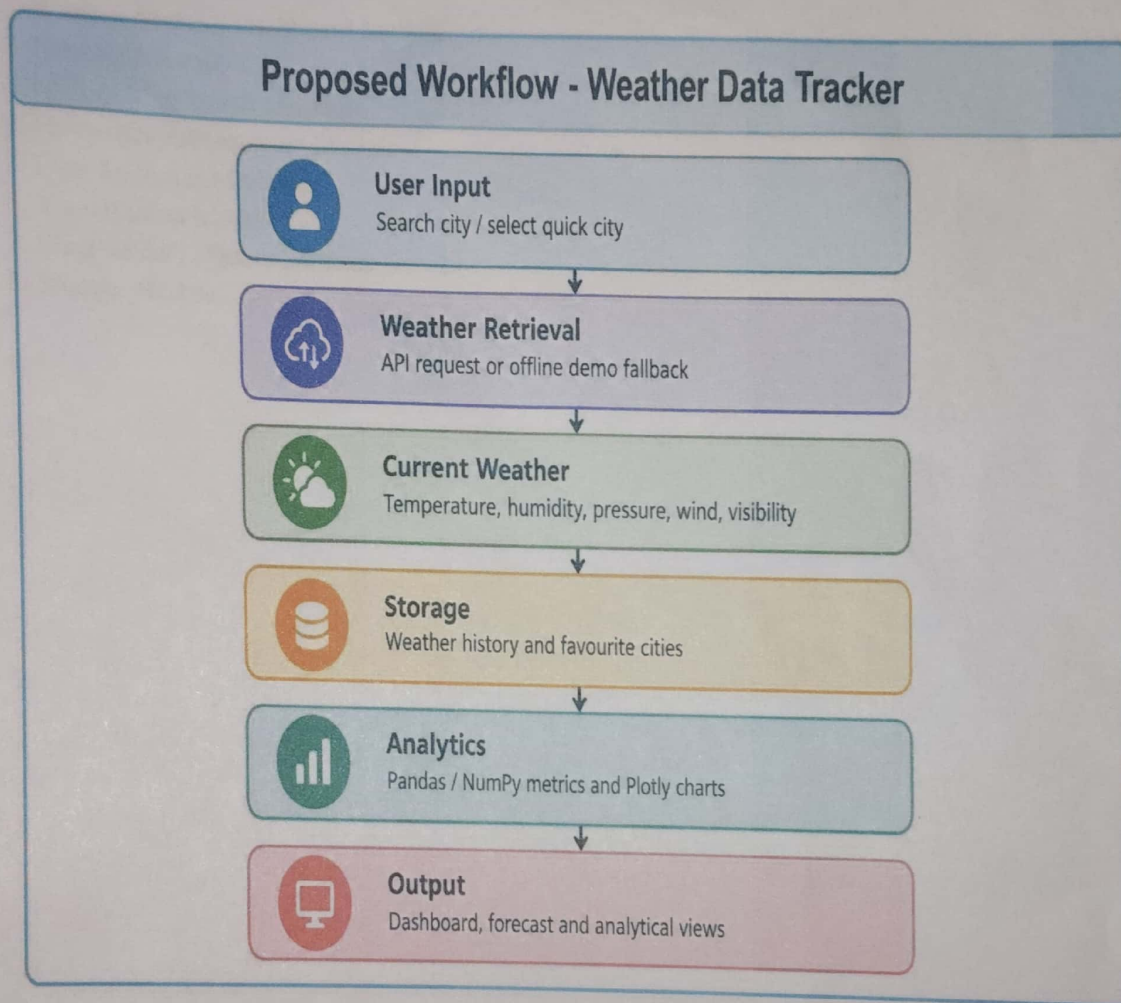


Figure 6.1 – proposed workflow

The standalone HTML file supplied with the project provides a self-contained demonstration interface. It uses predefined city data and generated fallback values when a city is not present in the known-city list, while localStorage maintains demo history and favourites.

Proposed Workflow

The proposed workflow begins when the user enters a city name. Weather information is then obtained from the configured service or demo fallback. The current values are displayed, and the search can be stored for later analysis. The user can move between forecast, history, favourites, and analytics modules.

Main Modules

1. Weather Search / Dashboard Module
2. Forecast Module
3. History Management Module
4. Favourites Management Module
5. Data Analytics Module
6. Visualisation Module
7. Weather API / Demo Fallback Service
8. Storage Module

7. DATASET DESCRIPTION

The weather dataset is formed from weather observations returned by the weather service or generated by the demo fallback. In the documented full-stack project, the records are stored in an SQLite database. The README identifies the database file as weather.db and the SQLAlchemy models WeatherHistory and FavoriteCity.

The standalone HTML implementation initialises a set of predefined city records and stores search history and favourites in browser localStorage. The predefined examples include Ahmedabad, Surat, Mumbai, Delhi, London and New York. These values are demonstration data embedded in the supplied application source.

Because weather conditions change continuously and the standalone demo contains fixed sample values for predefined cities, the fixed values should be interpreted as demonstration records rather than live historical observations.

DATASET DESCRIPTION (CONTINUED)

The project can work with the following logical data groups:

Table 7.1 – Logical Data

Data Group	Purpose
Current Weather	Stores the latest searched weather values for a city.
Forecast	Stores/display forecast values for multiple days.
Weather History	Maintains previous searches for analysis and review.
Favorite Cities	Maintains cities selected for quick access.
Analytics Metrics	Derived values such as averages, maximums and counts.

The dataset can therefore be treated as a collection of city-level weather observations containing numerical, categorical and time-related attributes.

8. DATASET ATTRIBUTES

Table 8.1 - Attributes

Attribute	Description
city	City name
country	Country code
temp	Current temperature in Celsius in the stored demo data
feels_like	Feels-like temperature
hum	Relative humidity percentage
pressure	Atmospheric pressure in hPa
wind	Wind speed in m/s
wind direction	Wind direction in degrees
clouds	Cloudiness percentage
vis	Visibility in kilometres
cond	Weather condition
desc	Weather description
sunrise / sunset	Sunrise and sunset times
time / timestamp	Time associated with the search record

Example Weather Records

Table 8.2 – ex. Weather Records

City	Temperature	Humidity	Condition	Wind Speed
Ahmedabad	32.5 °C	65%	Clear	3.6 m/s
Surat	31.0 °C	70%	Clouds	4.1 m/s
Mumbai	29.8 °C	78%	Haze	4.8 m/s
Delhi	28.2 °C	60%	Clouds	2.5 m/s
London	16.4 °C	82%	Rain	5.2 m/s
New York	21.0 °C	55%	Clear	3.9 m/s

9. Data Collection Process

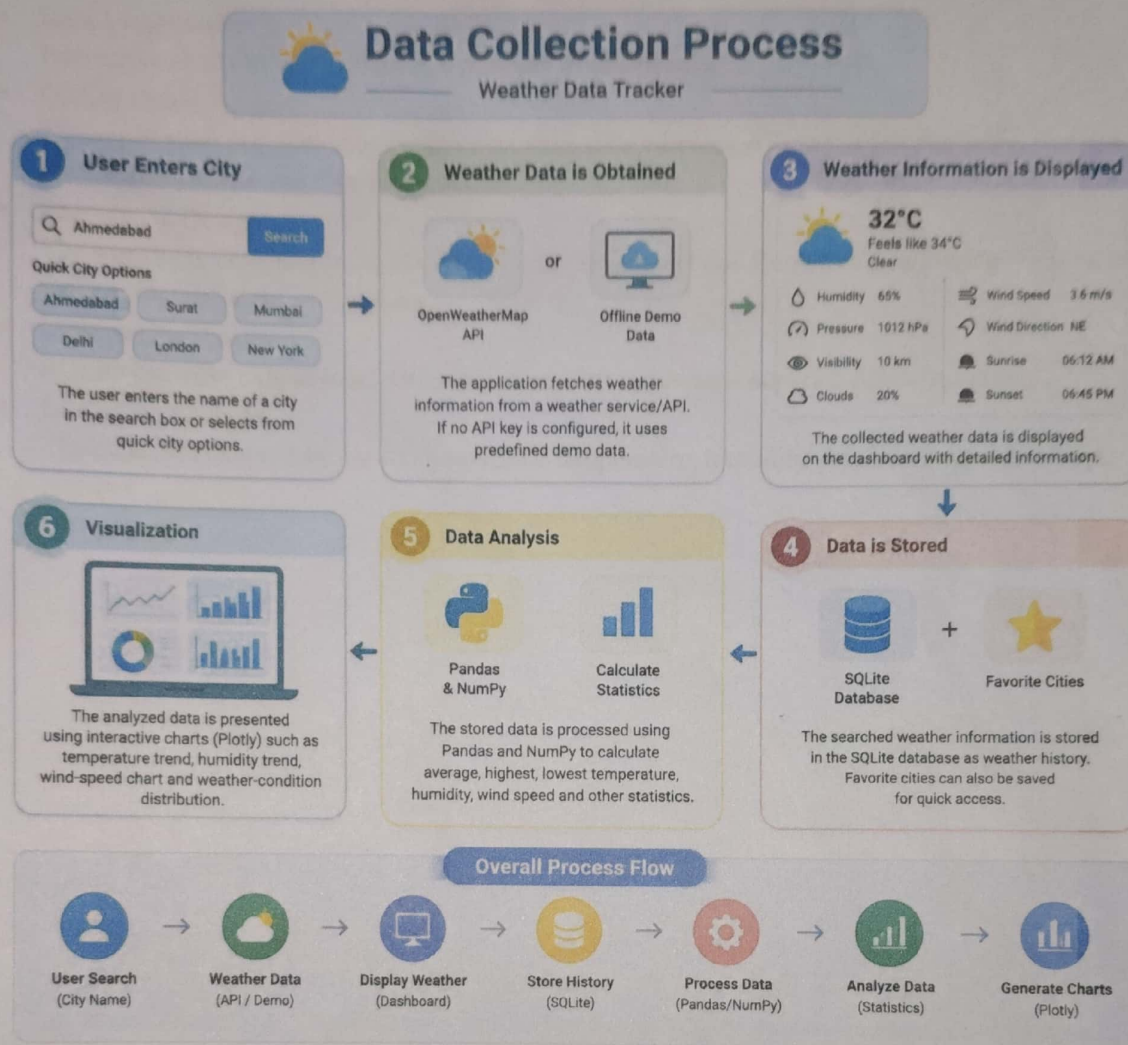


Figure 9.1 – Data Collection Process

Advantages

- **Easy to Use:**
The website provides a simple and user-friendly interface for searching weather information.
- **Real-Time Weather Information:**
The system can obtain weather information for the selected city through a weather service/API.
- **Weather History:**
Search records can be stored and managed for future reference.

- Favourite Cities:
Users can save frequently checked cities and access their weather information quickly.
- Data Analysis:
Weather history can be analysed to calculate average, highest and lowest temperatures, humidity and other statistics.
- Data Visualisation:
Interactive charts make weather trends easier to understand.
- Offline Demo Support:
The application provides sample/demo weather data when an API key is not available, making it useful for demonstrations and viva presentations.
- Responsive Design:
The website is designed using HTML5, CSS3, JavaScript and Bootstrap 5, allowing the interface to work across different screen sizes.
- Lightweight Database:
SQLite provides simple local data storage without requiring a separate database server.
- Useful for Analysis:
The collected data can be used to understand temperature, humidity, wind and weather-condition patterns.

10. DATA PREPROCESSING

Data preprocessing is performed before analysis so that weather records are consistent and suitable for calculations and visualisation.

Steps

1. Data loading
2. Missing-value checking
3. Duplicate removal
4. Text/value validation
5. Data type checking
6. Unit normalisation
7. Feature creation

Missing Values

Weather services may return incomplete values in some situations. The processing layer should identify missing fields before calculating averages or generating charts.

```
# Conceptual Pandas check
weather.isnull().sum()
```

Remove Duplicates

Duplicate weather records are identified and removed when required. This prevents the same weather observation from being counted multiple times during analysis.

```
weather = weather.drop_duplicates()
```

Text / Categorical Cleaning

City names, weather conditions and country codes should be normalised before grouping or counting.

```
weather['city'] = weather['city'].fillna("").str.strip()
weather['cond'] = weather['cond'].fillna("").str.strip()
```

Unit Handling

Unit handling is an important part of preprocessing weather data because different weather parameters are measured using different units.

The Weather Data Tracker mainly uses the following units:

Table 10.1 – Unit Handling

Weather Parameter	Unit
Temperature	°C / °F
Feels-like Temperature	°C / °F
Humidity	%
Atmospheric Pressure	hPa
Wind Speed	m/s
Visibility	km
Cloud Coverage	%

Temperature Conversion:

The application supports temperature display in both **Celsius (°C)** and **Fahrenheit (°F)**. The user can switch between the two units using the temperature unit option on the website.

The conversion formulas are:

Celsius to Fahrenheit:

$$^{\circ}\text{F} = (^{\circ}\text{C} \times 9/5) + 32$$

Fahrenheit to Celsius:

$$^{\circ}\text{C} = (^{\circ}\text{F} - 32) \times 5/9$$

Other weather parameters such as humidity and cloud percentage are represented as percentages, while pressure, wind speed and visibility use their respective standard units.

Proper unit handling ensures that weather values are displayed consistently and that calculations and visualisations are performed correctly.

11. METHODOLOGY

Methodology Steps:

Step 1: Data Collection

Collect weather data for the city entered by the user using the Weather API or demo data.

Step 2: Data Storage

Store the collected weather records in the SQLite database (weather.db) using SQLAlchemy.

Step 3: Data Preprocessing

Clean the collected data by:

- Handling missing values
- Removing duplicate records
- Cleaning city names and weather conditions
- Handling °C / °F units

Step 4: Exploratory Data Analysis (EDA)

Analyse the stored data using Pandas and NumPy to find:

- Average temperature
- Highest and lowest temperature
- Average humidity
- Average wind speed
- Total searches
- Unique cities

Step 5: Data Visualisation

Create charts using Plotly, such as:

- Temperature trend
- Humidity trend
- Wind speed
- Min vs. Max temperature
- Weather condition distribution

Step 6: Result Presentation

Display the analysed results, weather information, history, forecasts and charts on the web dashboard.

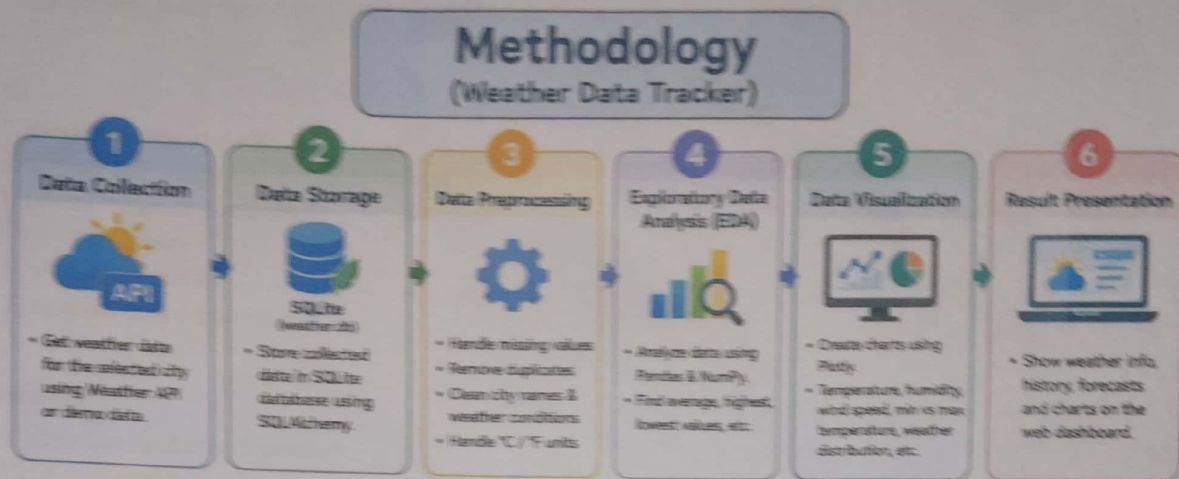


Figure 11.1 - Methodology

12. SYSTEM DESIGN

The system is designed as a web-based weather-data pipeline.

Input

- City name/user search
- Weather API response or demo record
- Current weather parameters
- Forecast values
- Historical records
- Favourite-city selections

Processing

- Validation and transformation
- Unit conversion
- History/favorite management
- Descriptive statistics
- Trend preparation
- Chart generation

Output

- Current weather dashboard
- 5-day forecast cards
- Search-history table
- Favourite-city cards
- Analytical KPI values
- Interactive Plotly charts

Data flow diagrams illustrate how search parameters, API payloads, database records, and chart JSON objects pass through the application boundary.

LEVEL 0 DATA FLOW DIAGRAM (CONTEXT DFD)

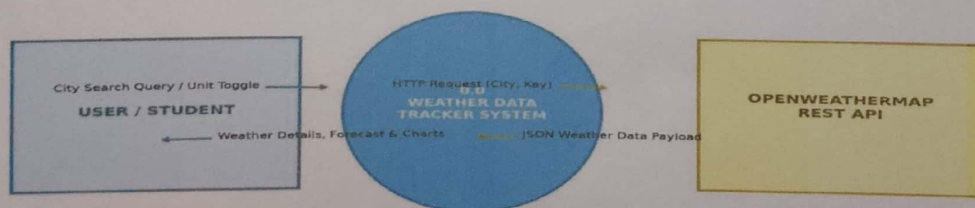


Figure 12.1 – Level 0 Flow Diagram

LEVEL 1 DATA FLOW DIAGRAM (DETAILED DFD)

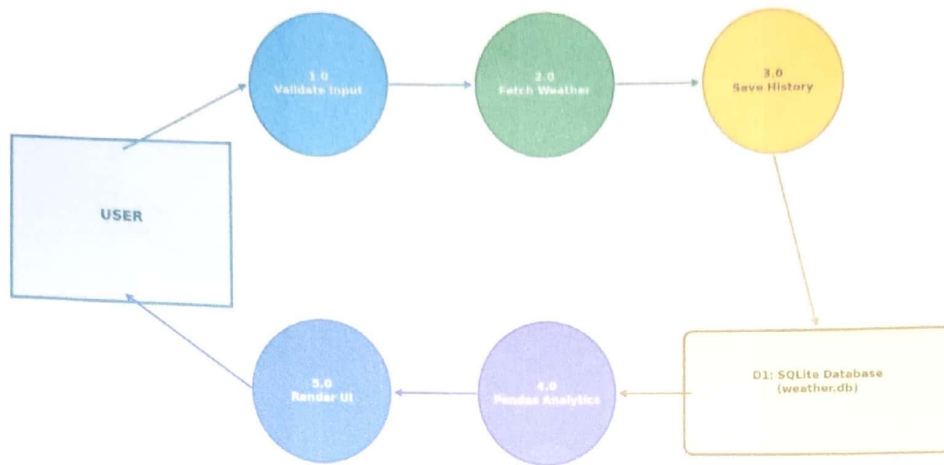


Figure 12.2 – Level 1 Flow Diagram

13. SYSTEM ARCHITECTURE

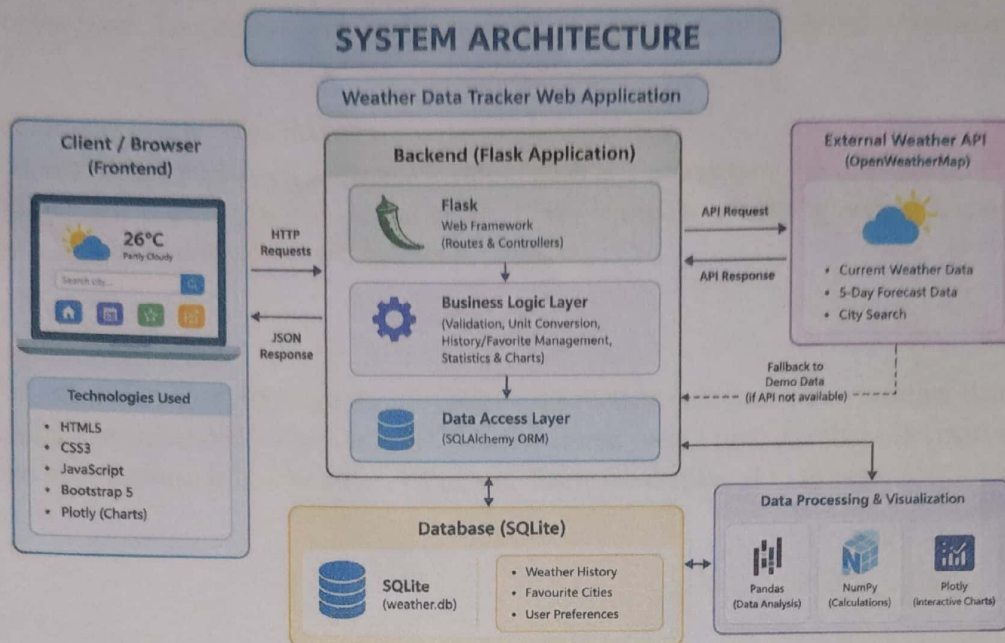


Figure 13.1 - System Architecture

The Weather Data Tracker follows a layered web-application architecture. The system consists of the Client/Browser, Backend, External Weather API, Database, and Data Processing & Visualisation components.

1. Client / Browser (Frontend):

The user interacts with the web application through the browser. The frontend allows users to search for cities and view current weather, forecasts, history, favourite cities, and analytical charts. It uses HTML5, CSS3, JavaScript, Bootstrap 5, and Plotly.

2. Backend:

The backend manages the main application logic. It receives requests from the frontend, validates the input, performs unit conversions, manages history and favourite cities, calculates statistics, and prepares data for visualisation.

3. External Weather API:

The system can communicate with an external weather service to obtain current weather and forecast information. The backend sends an API request and receives the weather data as a response. When API data is unavailable, the application can use demo weather data.

4. Database / Data Storage:

Weather history, favourite cities, and related records can be maintained through the data-storage layer. This allows previously searched information to be accessed and managed.

5. Data Processing & Visualisation:

Weather data is processed to calculate values such as average temperature, maximum temperature, humidity statistics, and trends. Plotly is used to generate interactive charts for better understanding of the weather data.

6. Overall Data Flow:

The user enters a city through the browser → the request is processed → weather data is obtained or generated → data is processed and stored → the final weather information, forecasts, history, favourite cities, KPIs, and charts are displayed to the user.

14. EXPLORATORY DATA ANALYSIS

EDA is used to understand weather-search records before presenting analytical results. Useful EDA dimensions include temperature, humidity, wind speed, weather condition, city frequency and search frequency.

The standalone source contains predefined demonstration values for Ahmedabad, Surat, Mumbai, Delhi, London and New York. These records can be used to demonstrate comparative charts without requiring an API key.

Exploratory Data Analysis – Sample Demonstration

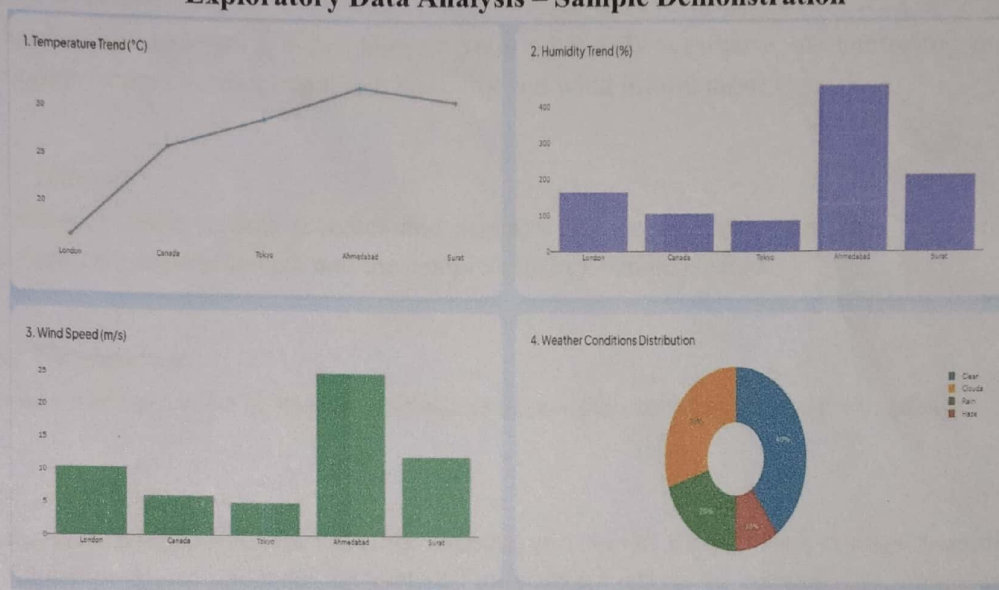


Figure 14.1 – Data Analysis

The charts above are derived from the fixed demonstration records embedded in the uploaded HTML source. They illustrate how the same weather attributes can be compared visually. They are not a claim about current real-world weather conditions.

15. RESULTS AND DISCUSSION

The developed Weather Data Tracker successfully demonstrates the major functional stages of a weather-data web application. The result is a unified interface where weather retrieval, presentation, storage and analytics are connected.

Result 1: Dashboard

The dashboard displays city, weather condition, temperature, feels-like temperature, humidity, pressure, wind speed, cloudiness, visibility, wind direction, sunrise and sunset information.

Result 2: Forecast

The application provides a 5-day forecast view with daily weather cards containing minimum and maximum temperature, weather condition, humidity and wind information.

Result 3: History

The history module records searches and displays city, temperature, humidity, condition and time. Individual records can be removed, and the complete history can be cleared.

Result 4: Favourites

Users can maintain a list of favourite cities and access their weather information quickly.

Result 5: Analytics

The analytics module calculates average temperature, highest temperature, average humidity and total searches. The source also generates interactive Plotly visualisations for temperature, humidity, wind and weather conditions.

Result 6: Reliability

The documented application includes an offline demo fallback when the OpenWeatherMap API key is not configured. This supports reliable classroom demonstrations.

Result 7: Unit Conversion

The standalone implementation supports a Celsius/Fahrenheit switch. The underlying temperature remains in Celsius and the display is converted when °F is selected.

Result 8: Responsive Interface

The application is designed to work across desktop, laptop, tablet and mobile browser widths. The navigation, search field, cards and forecast sections adapt to smaller screens.

16. FUTURE SCOPE

The Weather Data Tracker can be further improved with the following features:

- **Live Weather API:**
Integrate a real-time weather API to provide continuously updated weather information.
- **Extended Forecast:**
Provide forecasts for 7, 10, or 14 days.
- **Weather Alerts:**
Add notifications for heavy rain, storms, high temperature, and other weather conditions.
- **Advanced Analytics:**
Include historical weather trends and more statistical analysis.
- **Interactive Maps:**
Display weather conditions of different cities on an interactive map.
- **Mobile Application:**
Develop an Android or iOS version for easier access.
- **Cloud Database:**
Store weather history securely online and synchronize it across devices.
- **AI-Based Prediction:**
Use machine-learning techniques for advanced weather prediction and trend analysis.

These enhancements can make the Weather Data Tracker more accurate, intelligent, scalable, and useful for real-time weather monitoring.

CONCLUSION

The Weather Data Tracker Web Application demonstrates the practical integration of web development and Data Science concepts. The project provides a single interface for searching weather information, viewing forecasts, maintaining history, organising favourite cities and analysing stored weather records.

The README documents a full-stack technology stack consisting of Python 3, Flask, SQLite, SQLAlchemy, Pandas, NumPy, Plotly, Matplotlib, HTML5, CSS3, JavaScript and Bootstrap 5. The uploaded standalone HTML implementation demonstrates the front-end experience, browser-side history/favorites and interactive visualization behavior.

The system successfully addresses the project objectives by presenting weather information in a structured format and adding analytical features to a normal weather lookup workflow.

The application also demonstrates the importance of fallback handling. When a live API key is unavailable, the demo mode can still provide representative weather records, which makes the application suitable for academic demonstration and viva presentation.

Overall, the project shows how weather data can be collected, organised, processed and visualised through a responsive web application.